

Integration of OpenRadioss with Open-Source Structural Optimization Frameworks

Introduction to Multidisciplinary Design Optimization in Explicit Dynamics

The optimization of structures subjected to highly nonlinear, transient dynamic events represents one of the most computationally demanding frontiers in modern computational mechanics. Engineering scenarios such as vehicular crashworthiness, blast and ballistic impacts, electronic drop testing, and highly distortive fluid-structure interactions require solvers capable of resolving large deformations, complex multi-body contact algorithms, and severe material nonlinearities¹. For decades, these domains were exclusively served by proprietary, closed-source finite element analysis (FEA) solvers. This proprietary paradigm severely bottlenecked the development of highly integrated, automated Multidisciplinary Design Optimization (MDO) pipelines, as the integration of commercial solvers into high-throughput design loops often incurred prohibitive software licensing costs and restricted access to lower-level programmatic interfaces³.

In September 2022, the landscape of dynamic simulation was fundamentally altered when Altair released OpenRadioss, an open-source version of its industry-proven explicit dynamics solver⁵. Operating under the AGPL-3.0 license, OpenRadioss utilizes the exact same input formats, source code, and time-integration schemes as its commercial counterpart, enabling academic institutions, research laboratories, and enterprise engineering teams to integrate a world-class explicit solver into open-source optimization frameworks⁵. The ability to download pre-built binaries or compile the solver from source code allows engineers to deploy OpenRadioss across massive, highly parallelized cloud computing architectures without license limitations¹.

However, integrating OpenRadioss into an automated optimization loop presents distinct programmatic and mathematical challenges. Unlike implicit FEA solvers, which evaluate static or slowly evolving phenomena using matrix inversion and Newton-Raphson iterations, OpenRadioss utilizes explicit time integration. This method calculates the kinematic state of a system at sequential, infinitesimal time increments determined by the Courant-Friedrichs-Lewy (CFL) stability condition². The dynamic, highly discontinuous nature of explicit simulations—characterized by element deletion, transient shockwaves, and shifting contact interfaces—results in highly noisy response surfaces. Furthermore, OpenRadioss does not natively possess an internal gradient-based optimizer or an adjoint solver capable of exporting analytic design sensitivities⁹. Consequently, OpenRadioss must be meticulously coupled with external optimization software using "black-box" wrapper methodologies.

This comprehensive report elucidates the exhaustive architectural strategies, programmatic

wrappers, and high-performance computing (HPC) methodologies required to seamlessly couple OpenRadioss with prominent open-source structural optimization frameworks, including OpenMDAO, Dakota, pyMOTO, and PyLCSS. It further explores the mitigation of explicit computational bottlenecks through the deployment of machine learning surrogates and advanced time-step controls.

The Architecture of OpenRadioss for Automated Workflows

Before an external optimization algorithm can successfully orchestrate an OpenRadioss simulation, the solver's fundamental input and output (I/O) architecture must be thoroughly understood. Because automated optimizers function by perturbing design variables and reading the resulting structural response, the optimization wrapper must be capable of programmatically reading, mutating, and writing OpenRadioss formats without manual intervention.

Source Compilation and Precision Tuning

Optimization workflows often require thousands of sequential or parallel simulation runs, making execution speed paramount. OpenRadioss provides various compilation targets tailored to specific hardware architectures. Compiling the solver directly from the source code hosted on GitHub allows optimization engineers to fine-tune the executable for the specific computing environment, such as utilizing the Intel oneAPI HPC Toolkit for optimal Fortran and C compiler performance on x86 architectures, or employing the armflang compiler for AWS Graviton cloud instances⁷.

The build architecture significantly impacts the optimization loop's turnaround time. OpenRadioss provides both double-precision (default 64-bit) and extended single-precision (32-bit, designated with the `_sp` suffix) executables¹³. For optimization studies where highly localized stress accuracy is secondary to global energy absorption or peak intrusion—such as early-stage conceptual design—deploying the `starter_linux64_gf_sp` and `engine_linux64_gf_sp` binaries reduces memory bandwidth requirements and accelerates the floating-point operations¹³. Testing indicates that executing models without heavy contact penalties in single precision can yield a computational speed increase of approximately thirty percent, directly accelerating the optimization algorithm's convergence time¹⁴.

Input Deck Anatomy and Parametrization

OpenRadioss partitions simulation definitions into two primary ASCII files, adhering to a strict naming convention that an optimization wrapper must navigate:

1. **The Starter File (*_0000.rad):** This file defines the spatial domain. It contains the fundamental mesh geometry, including node coordinates and element connectivity. It also houses the material laws (e.g., piecewise linear plasticity, elastoviscoplasticity), property cards (e.g., shell element thicknesses, beam cross-sections), boundary conditions, and initial velocity vectors³.
2. **The Engine File (*_0001.rad, *_0002.rad):** This file defines the temporal domain. It

specifies the termination time, time integration controls, mass scaling parameters, penalty formulations for contact interfaces, and the frequency of data output requests³. Because OpenRadioss is fundamentally a compiled Fortran executable, its native text parser expects rigid, twenty-character column widths for numeric data fields across its hundreds of keywords¹⁵. When an optimization algorithm perturbs a design variable—such as substituting a new shell thickness or yield strength into the _0000.rad deck—the automated script must meticulously preserve this strict column formatting. If a parametrically generated float value expands beyond the twenty-character limit or shifts adjacent data, the OpenRadioss Starter will fail to parse the deck, resulting in a "BAD CARD" error and terminating that specific optimization branch¹⁵.

To circumvent fragile, regex-based text manipulation, the OpenRadioss community has developed robust programmatic parsers. The `operator` Python package, for instance, provides object-oriented data classes for OpenRadioss keywords¹⁶. This allows automated Python scripts to instantiate objects representing specific cards (e.g., /NODE, /BEAM, or /MAT), manipulate them using numerical arrays from libraries like NumPy, and safely serialize the output back into the strictly formatted text required by the solver¹⁶. This programmatic approach ensures that as an optimizer explores vast, multi-dimensional design spaces, the resulting input decks remain syntactically valid.

Output Formatting and Data Extraction

The critical secondary phase of an optimization loop involves extracting the simulation response to evaluate the objective function and constraint parameters. OpenRadioss generates a suite of file types that the optimization wrapper must parse:

- **Time-History Files (*T01, *T02):** These ASCII files record global scalar data over the duration of the impact event, such as the total internal energy, kinetic energy, the reaction force on a rigid wall, or the specific displacement trajectory of an instrumented node³.
- **Animation Files (ANIM/*):** These ASCII files capture the full-field state of the mesh (stress tensors, plastic strains, displacements) at specified engine intervals³.
- **H3D Files:** A proprietary Altair binary format containing highly compressed full-field results³.

For open-source optimization frameworks, proprietary binary formats pose significant extraction barriers. Consequently, automated workflows predominantly rely on the T01 files for scalar objectives—such as minimizing the peak deceleration of a vehicle occupant—and the ANIM files for field-level constraints, such as ensuring no structural element exceeds a predefined plastic strain failure threshold.

The community-developed Vortex-Radioss library is instrumental for this data extraction phase. Released under the Mozilla Public License, this open-source Python tool enables direct programmatic reading of OpenRadioss outputs¹⁸. It provides a `RadiossTHReader` class that unpacks the dense T01 data arrays, allowing the optimizer to extract transient scalar curves into standard Python dictionaries¹⁸. Furthermore, its `RadiossReader` class can parse animation files, while the `Anim_to_D3plot` module converts OpenRadioss animation formats into the

widely supported LS-DYNA d3plot format, facilitating cross-compatibility with post-processing routines built for legacy solvers¹⁸. Alternatively, built-in OpenRadioss command-line utilities such as `anim_to_vtk` and `th_to_csv` can be executed within the optimization driver script to convert the raw outputs into formats seamlessly ingested by data science libraries like Pandas or PyVista¹³.

High-Performance Integration with OpenMDAO

OpenMDAO is a highly advanced, open-source high-performance computing framework developed by NASA, explicitly designed for Multidisciplinary Design Optimization²⁰. Written entirely in Python, its architecture utilizes the Modular Analysis and Unified Derivatives (MAUD) methodology to manage complex data flows, compute coupled derivatives, and orchestrate massive system-of-systems decomposition²².

Encapsulation via ExternalCodeComp

To integrate OpenRadioss into an OpenMDAO Problem and Group hierarchy, the explicit solver must be encapsulated within an ExternalCodeComp object²³. OpenMDAO cannot hold OpenRadioss in active memory; therefore, the ExternalCodeComp is specifically engineered to run the compiled Fortran executable as a subprocess on the host operating system. This component handles the unique challenges of executing heavy computational jobs, including execution delays, environment variable management, and asynchronous polling.

In Python, the optimization engineer creates a subclass of ExternalCodeComp. Within the component's `setup()` method, the `add_input()` and `add_output()` functions are invoked to declare the specific design variables interacting with OpenRadioss (e.g., shell thickness, geometry scale) and the required responses (e.g., peak intrusion, total energy absorption)²⁴. The execution is triggered by defining the command option, which issues the shell commands to execute the wrapper script.

OpenMDAO ExternalCodeComp Configuration Option	Expected Type	Functional Description for OpenRadioss Integration
command	List or String	Specifies the command line to execute the analysis driver or the OpenRadioss executables directly via subprocess polling ²⁴ .
external_input_files	List	A list of files (e.g., the mutated _0000.rad deck) that OpenMDAO verifies must exist before

		execution, preventing failed shell calls ²⁴ .
external_output_files	List	A list of files (e.g., the T01 time-history file) that must exist after execution. If OpenRadioss crashes due to instability, these files will not generate, triggering an error state ²⁴ .
fail_hard	Boolean	If set to True, an OpenRadioss failure raises a hard RuntimeError. If False, it raises a soft AnalysisError, allowing the optimizer to intelligently retreat from the unstable region of the design space ²⁴ .
timeout	Float	The maximum allowable time for the OpenRadioss subprocess to complete. Critical for preventing infinite hangs if the explicit solver encounters a highly distortive element that causes time-step collapse ²⁴ .
env_vars	Dictionary	Used to pass critical HPC environment variables, such as OPENRADIOSS_ROOT, OMP_NUM_THREADS, and LD_LIBRARY_PATH required by the MPI and OpenMP backend ¹⁵ .

Intelligent File Wrapping Utilities

Because the ExternalCodeComp relies entirely on file-based communication, OpenMDAO provides native utilities to facilitate the required text manipulation. The InputFileGenerator class

acts as a templating engine. The engineer provides a base OpenRadioss starter file with anchor points replacing the standard twenty-character numerical columns. During each iteration, OpenMDAO injects the current floating-point values from the active design variables into these anchors, securely generating a syntactically valid _0000.rad input deck²³.

Once the OpenRadioss engine terminates, OpenMDAO utilizes the FileParser utility. The explicit output files contain thousands of lines of text logging cycle progression, energy balances, and mass scaling metrics. The FileParser utilizes an anchoring methodology, searching the output text for specific string fragments—such as "MAXIMUM KINETIC ENERGY"—and systematically extracting the required numerical fields²³. Explicit solvers are prone to severe numerical instabilities during aggressive optimization routines; if a highly distorted element causes a floating-point overflow, the OpenRadioss output may write NaN or Inf into the text fields. The OpenMDAO FileParser is natively designed to recognize these standard Python built-in values, allowing the framework to catch numerical overflows and take corrective action rather than suffering a catastrophic framework crash²³.

Derivative Methodologies and Solvers

Explicit dynamic simulations present severe theoretical challenges for gradient computation. Because crash events involve highly discontinuous physical phenomena—such as complex node-to-surface contact algorithms, element deletion upon material failure, and piecewise linear plasticity—the objective function's response surface is inherently non-smooth and exceptionally noisy²⁵. Standard gradient-based optimization algorithms, such as Sequential Least Squares Programming (SLSQP), struggle to navigate these spaces because numerical noise heavily corrupts finite-difference gradient approximations.

While OpenMDAO fully supports the declaration of finite-difference derivatives (via `self.declare_partials(of='*', wrt='*', method='fd')`) for any component²⁶, executing forward finite-differences with an explicit solver is computationally devastating. Calculating a simple

forward gradient requires $N + 1$ full OpenRadioss simulations per optimization iteration,

where N is the number of design variables²¹. Consequently, engineers coupling OpenRadioss with OpenMDAO typically bypass gradient-based drivers entirely. Instead, they leverage OpenMDAO's derivative-free optimizers, such as the ScipyOptimizeDriver utilizing the COBYLA algorithm, or deploy OpenMDAO's robust Design of Experiments (DOE) generators to uniformly sample the design space and build continuous surrogate models prior to optimization²⁴.

The versatility of OpenMDAO is demonstrated in massive aerospace integration projects, where external solvers (like OpenVSP for geometry and FLOPS for system weight) are coupled into unified workflows predicting aero-structural loads²⁷. OpenRadioss fits seamlessly into this architecture, allowing a high-fidelity explicit dynamics component (evaluating, for example, a bird strike on a composite wing leading edge) to act as an implicit constraint within a broader, multidisciplinary aircraft sizing optimization loop²⁷.

Integration with the Dakota Toolkit

Dakota, developed by Sandia National Laboratories, is a premier C++ software toolkit

engineered for systems analysis, parameter estimation, uncertainty quantification (UQ), and optimization²⁹. Its robust, file-driven architecture is specifically tailored to wrap large-scale, high-fidelity engineering simulations that lack native internal optimizers, making it an ideal companion for OpenRadioss³⁰.

The Dakota Analysis Driver Interface

Unlike OpenMDAO, which wraps the external code in Python object classes, Dakota utilizes a strict, universally applicable file-exchange protocol orchestrated through a central input text file. Dakota interacts with OpenRadioss as a pure "black-box," possessing zero awareness of the explicit mechanics occurring within the solver³⁰. The communication relies on generating a structured params.in.X file and reading a corresponding results.out.X file³⁰.

A typical Dakota configuration file driving OpenRadioss specifies the core parameters: interface analysis_driver = 'radioss_execution_script.sh' parameters_file = 'params.in' results_file = 'results.out' file_tag file_save The radioss_execution_script.sh (or an equivalent Python script) serves as the interface glue³⁰. During an iteration, Dakota writes the current variable values to the parameter file. Crucially, this file contains the Active Set Vector (ASV), which dictates exactly what response Dakota requires the simulation to return—such as a flag of 1 requesting only the objective function value, or 2 requesting gradients³⁰. The analysis driver reads this parameter file, modifies the OpenRadioss _0000.rad file accordingly, executes the solver utilizing the host system's MPI libraries, and parses the resulting output³⁰. Finally, the driver writes the extracted quantities into the results.out file in a simple, flat format (e.g., 45.2 obj_fn) which Dakota ingests to compute the subsequent iteration³⁰.

Optimization Algorithms and Uncertainty Quantification

Dakota's vast internal library of algorithms allows engineers to apply diverse mathematical methodologies to OpenRadioss simulations:

1. **Derivative-Free Optimization:** Acknowledging the chaotic, non-differentiable nature of crash and blast simulations, Dakota avoids gradient-based methods like conmin_frcg or dot_mmfd when coupled with explicit solvers³¹. Instead, the toolkit deploys Pattern Search (PS), Evolutionary Algorithms (EA), or Surrogate-Based Optimization (SBO) to navigate toward global optima without requiring analytic or finite-difference gradients³².
2. **Uncertainty Quantification (UQ):** OpenRadioss is predominantly utilized for safety-critical evaluations—such as the drop testing of consumer electronics, or the crashworthiness of automotive battery enclosures—where manufacturing tolerances and material variability introduce significant real-world uncertainty¹. Dakota excels in wrapping OpenRadioss within Monte Carlo (MC) or Latin Hypercube Sampling (LHS) routines³¹. By defining parameters like yield strength and shell thickness with statistical distributions (using keywords such as uniform_uncertain or normal_uncertain), Dakota autonomously executes hundreds of OpenRadioss runs³¹. Post-processing this batch of outputs allows engineers to rigorously quantify the probability of structural failure or calculate the standard deviation of peak loads, moving beyond deterministic design³².

3. **Global Sensitivity Analysis:** Utilizing methods analogous to those applied to massive multiphysics nuclear codes like DYMOND or SAS4A, Dakota can execute OpenRadioss across a parameter space to compute Sobol indices³². This sensitivity analysis identifies not only which individual design variables dictate overall structural performance, but also uncovers the synergistic, non-linear interactions between coupled variables³³.

Advanced Optimization: Topology and Shape Design

While parametric sizing optimization—such as iteratively tuning metal gauges or composite ply properties—is efficiently handled by frameworks like OpenMDAO and Dakota, topology and shape optimization require specialized mathematical architectures. Topology optimization seeks to determine the optimal layout of material within a predefined design space to maximize performance while minimizing mass.

Challenges of Topology Optimization in Explicit Dynamics

Topology optimization in linear static analysis relies heavily on the Solid Isotropic Material with Penalization (SIMP) method and adjoint sensitivity analysis. However, implementing SIMP directly within OpenRadioss is mathematically and programmatically harrowing. The explicit

time step size Δt is strictly governed by the CFL condition, ensuring that the time step remains smaller than the time required for a stress wave to traverse the smallest characteristic dimension of any element in the mesh:

$$\Delta t \leq \frac{l_c}{c}$$

where l_c is the element characteristic length and c is the speed of sound in the material. In a standard SIMP approach, as elements are penalized by the optimizer and their density approaches zero, their stiffness-to-mass ratio can fluctuate violently. This often leads to vanishingly small explicit time steps or instantaneous element inversions, immediately crashing the solver. Furthermore, objective functions like plastic energy absorption under dynamic crushing loads are inherently path-dependent and non-conservative, making adjoint derivative calculations exceptionally complex and generally unavailable in open-source formats¹⁰.

Integration with pyMOTO

To bridge explicit dynamics with topology optimization, researchers employ gradient-free methods or modular surrogate gradients. pyMOTO is an open-source Python library designed to provide a highly modular and easily reconfigurable framework for topology optimization¹⁰. pyMOTO decomposes complex optimization problems into reusable mathematical building blocks—such as density filters, finite-element assembly routines, and linear solvers—while automatically computing partial sensitivities using chain rule backpropagation³⁵.

To couple pyMOTO with OpenRadioss, the explicit solver is treated as a highly specialized, nonlinear evaluation module within the broader network. Because exact analytical derivatives

from the OpenRadioss transient analysis are not exposed, engineers can leverage pyMOTO's robust SIMP architecture alongside heuristic, localized finite-difference, or surrogate-based gradients to iteratively update the voxel density field based on the explicit crash response. Upon reaching convergence, pyMOTO facilitates the direct export of the optimized geometry into standard STL or OBJ formats³⁶. This geometric output can subsequently be re-meshed and subjected to a final, high-fidelity verification run in OpenRadioss to certify crashworthiness performance.

PyLCSS: Unified Low-Code Integration

The formidable barrier of scripting the integration between sophisticated optimization algorithms and robust explicit solvers is effectively eliminated by PyLCSS (Python Low-Code System Solutions)³⁷. PyLCSS is a comprehensive product development environment featuring an intuitive, node-based visual interface that seamlessly connects various disparate open-source engineering tools into unified pipelines³⁷.

The PyLCSS platform natively integrates:

- **OpenRadioss:** For explicit crash and impact simulation, coupled with integrated VTK animated result playback³⁶.
- **CalculiX:** For linear and static structural FEA evaluations³⁶.
- **pyMOTO:** For voxel-based SIMP topology optimization³⁶.
- **Optimization Solvers:** A suite of multi-objective algorithms including NSGA-II, SLSQP, COBYLA, and Differential Evolution³⁶.
- **Parametric CAD:** Integration with CadQuery and FreeCAD via a live Python bridge, allowing continuous geometry regeneration³⁶.

A distinguishing paradigm of PyLCSS is its implementation of the "Solution Space" approach for robust design. Rather than forcing OpenRadioss to converge on a single, razor-thin optimal design point—which may ultimately fail in physical production due to manufacturing tolerances and material variances—PyLCSS drives the solver to identify vast, box-shaped regions of valid designs³⁶. The engineer wraps the OpenRadioss execution sequence inside a simple Python evaluate(x) function node. PyLCSS then orchestrates advanced multi-objective algorithms (like NSGA-II) to evaluate OpenRadioss dynamically across the entire design space. By mapping the highly nonlinear structural responses to functional safety requirements, PyLCSS allows engineers to establish decoupled subsystem development spaces, ultimately yielding a robust, impact-resistant product platform that accommodates manufacturing variance³⁶.

High-Performance Execution and Cloud Computing

Evaluating dynamic crash events is highly resource-intensive. A single, high-fidelity full-vehicle crash simulation in OpenRadioss can require hundreds of CPU cores and several hours to execute to completion³. Integrating this computational burden into an optimization loop requiring hundreds or thousands of evaluations necessitates highly scalable computing infrastructure.

Leveraging Scalable Cloud Resources

To overcome local hardware limitations and accelerate design iteration cycles, open-source frameworks are increasingly deployed on remote cloud high-performance computing (HPC) platforms, such as CloudHPC⁸. By uploading the fully prepared _0000.rad and _0001.rad input decks to virtual machines tailored with high-frequency CPUs and multi-core processors, engineers can drastically reduce simulation turnaround times⁸. Platforms like CloudHPC scale computing resources elastically, allowing an optimization driver like Dakota or OpenMDAO to launch multiple parallel instances of OpenRadioss simultaneously across different virtual machines, effectively parallelizing the optimization evaluation phase⁸. This capability is critical when executing comprehensive Design of Experiments (DoE) or mapping stochastic uncertainty, where independent simulation branches do not require real-time synchronization. Furthermore, cloud environments provide pre-configured operating systems and dependency management, which is crucial for reproducibility. To ensure that OpenRadioss executes reliably across massive clusters without encountering library conflicts, engineers employ containerization technologies such as Apptainer (formerly Singularity) or Docker³⁹. By defining an Apptainer .def file, the entire compilation process—including downloading the specific Git commit of OpenRadioss, installing Fortran compilers, and building Open MPI—is packaged into a single, immutable .sif image file¹². This containerized executable can be distributed across AWS Graviton cloud instances, guaranteeing that every node in the optimization loop executes the exact same binary environment¹².

Mitigating Computational Expense: Surrogates and Machine Learning

Despite the utilization of HPC clusters, directly inserting high-fidelity explicit simulations into an active optimization loop remains computationally intractable for vast parameter spaces. The industry standard has therefore pivoted toward surrogate-assisted optimization, wherein OpenRadioss acts purely as an offline data-generator prior to the rapid execution of the optimization routine.

PhysicsNeMo and Automated Dataset Generation

To train an accurate surrogate model, an extensive dataset capturing the underlying physics must first be generated. NVIDIA's PhysicsNeMo framework provides an end-to-end recipe specifically engineered for generating parameterized OpenRadioss datasets¹⁵. Utilizing Latin Hypercube Sampling (LHS), the framework mutates the _0000.rad starter deck across defined boundary and material parameters, launching heavily parallelized OpenRadioss runs. The PhysicsNeMo generator automates two primary explicit test cases:

Benchmark Flow	Physical Scenario	Perturbed Design Parameters	Default DoE Size
bumper_beam	Front-impact pole	Geometry scale	135 continuous

	test of a dual-phase steel bumper assembly.	(X,Y,Z multipliers), impactor initial velocity, shell thickness scaling, rigid-wall diameter, and rigid-wall geometric origin ¹⁵ .	runs ¹⁵ .
drop_test	Highly nonlinear cell-phone drop impact onto a rigid plane.	Young's modulus scaling across five distinct materials (polymer, battery, glass, PCB, composites), and rigid-wall plane orientation Euler angles (rx, ry, rz) ¹⁵ .	192 continuous runs ¹⁵ .

This automated pipeline manages the entire execution lifecycle: creating the input decks, launching the OpenRadioss engine via MPI, automatically converting the ANIM files to VTK or d3plot formats utilizing the Vortex-Radioss library, and finally restructuring the metadata into flat, machine-learning-loader-friendly JSON and CSV outputs¹⁵.

Mesh-Native Surrogate Models

The resulting dataset serves as the ground truth for training sophisticated machine learning models. While standard tabular surrogates (like Random Forests or Kriging algorithms, accessible within PyLCSS) can map scalar inputs directly to scalar outputs²⁵, modern optimization increasingly relies on "mesh-native" models. Architectures such as Graph Neural Networks (GNNs) or Transformer-based neural operators (Transolvers) are trained directly on the raw point-cloud geometry extracted from the converted VTK files³.

These advanced neural operators do not require handcrafted scalar features. Instead, by processing the raw geometry through mechanisms like slice-based attention, the model inherently learns underlying structural mechanics—such as spatial load distribution, bending stiffness gradients, and complex contact patch evolution³. The PLAID (Physics-Learning AI Data) model further standardizes this approach, releasing massive OpenRadioss-derived datasets (such as Tensile2d for nonlinear elastoviscoplasticity, and 2D_EIPIDynamics for time-dependent topological variation and fracture) to train highly generalized physics-informed models⁴¹.

Once the surrogate model is trained, it entirely replaces the OpenRadioss executable within the OpenMDAO or Dakota loop. Because the neural network can infer a full-field crash response in milliseconds rather than hours, the optimizer can deploy highly sample-inefficient, global search algorithms to explore millions of design points and locate true global optima. The final

optimal design candidate is subsequently passed back to OpenRadioss for a single, definitive verification run, confirming the surrogate's prediction³.

Accelerating Solvers via Advanced Mass Scaling (AMS)

If surrogate models cannot adequately capture the required nonlinear physics and the optimizer must evaluate OpenRadioss directly, the execution time per iteration must be drastically reduced. Traditional mass scaling (/DT/NODA/CST) prevents explicit time steps from crashing by physically adding mass to the smallest, most distortive elements in the mesh. However, this artificially inflates the kinetic energy of the system, potentially altering the dynamic response and invalidating the optimization results¹⁴.

Conversely, Advanced Mass Scaling (/DT/AMS) operates by altering only the non-diagonal terms of the mass matrix. It allows the explicit solver to artificially increase the time step without modifying the overall inertia effects on translational global acceleration⁴². For explicit optimization, injecting AMS commands into the _0001.rad file allows OpenRadioss to maintain computational stability at time steps up to ten times higher than conventional physical bounds.

Time Step Control Method	Mechanism of Action	Impact on Optimization Performance
No Control	Time step restricted by the smallest element characteristic length (CFL limit).	Exceedingly slow execution time; impractical for extensive optimization loops ¹⁴ .
Standard Scaling (/DT/NODA/CST)	Adds artificial mass to nodes to maintain a minimum time step.	Accelerates runtime by a factor of 3, but degrades physical accuracy due to altered kinetic energy ⁴² .
Advanced Mass Scaling (/DT/AMS)	Augments off-diagonal matrix terms, keeping total mass constant.	Reduces CPU time by a factor of 3 to 5 while maintaining high physical accuracy and correct dynamic response ¹⁴ .

By deploying AMS, engineers significantly accelerate the individual FEA evaluations without severely compromising accuracy. This reduction in overhead makes direct, gradient-free optimization loops utilizing Dakota or PyLCSS substantially more viable for complex, mid-fidelity structural models.

Conclusion

The integration of OpenRadioss with open-source structural optimization frameworks

represents a transformative democratization of advanced explicit dynamics analysis, bringing high-fidelity impact, crashworthiness, and ballistic evaluation capabilities to a broad spectrum of engineering disciplines. Due to the inherent lack of analytic sensitivities within explicit architectures, optimization relies fundamentally on robust "black-box" simulation coupling. This methodology hinges on translating parametric data arrays generated by the optimizer into strictly formatted OpenRadioss ASCII input decks, safely executing the solver across scalable cloud HPC environments, and subsequently utilizing advanced parsers like Vortex-Radioss to extract complex transient outputs.

The selection of the optimization framework dictates the structural approach to the engineering problem:

- **OpenMDAO** provides an exceptionally powerful, Pythonic hierarchical architecture, allowing engineers to wrap OpenRadioss securely within an ExternalCodeComp and orchestrate massive, coupled multidisciplinary systems leveraging automated file parsers.
- **Dakota** excels at managing high-throughput system calls via robust shell script analysis drivers, offering sophisticated, production-ready capabilities for derivative-free optimization, global sensitivity analysis, and statistical uncertainty quantification over highly noisy dynamic responses.
- **pyMOTO and PyLCSS** effectively map the explicit solver into user-friendly graphical environments, facilitating shape and topology optimization, and pioneering multi-objective solution space exploration to identify robust, manufacturing-tolerant designs.

Ultimately, because explicit time integration is inherently computationally punishing and yields chaotic response surfaces, the most powerful and effective optimization workflows are hybrid. By employing Latin Hypercube Sampling to generate extensive OpenRadioss datasets, deploying Advanced Mass Scaling to accelerate individual solver evaluations, and training Transolvers or Graph Neural Networks to act as ultra-fast surrogate models, engineers can execute exhaustive optimization studies over complex, violent dynamic events. This synthesis of open-source solvers, automated text parsers, and machine learning ensures that the optimization of safer, more efficient structures is no longer constrained by software licensing, but rather limited only by the scale of the available computing infrastructure.

Works cited

1. Simcenter Radioss software | Siemens,
<https://www.siemens.com/en-us/products/simcenter/mechanical-simulation/radioss/>
2. Altair Radioss | Crash Simulation Software & Explicit Dynamics Solver - TrueInsight, <https://www.trueinsight.io/altair/radioss>
3. Building a Transolver Model on OpenRadioss Data | by Sudeep Chavare - Medium, <https://medium.com/@sudeepdc/building-a-ground-up-transolver-on-openradioss-data-2fc2ded1fff4>
4. V2004-WP3.1: OpenRadioss as a reliable replacement for LS-DYNA, <https://openradioss.org/wp-content/uploads/2025/08/TNO-2024-R11057A.pdf>

5. OpenRadioss: Home Page, <https://openradioss.org/>
6. OpenRadioss is a powerful, industry-proven finite element solver for dynamic event analysis - GitHub, <https://github.com/OpenRadioss/OpenRadioss>
7. Compiling procedure to OpenRadioss. - foamBuilder, <https://www.foambuilder.com/pages/OpenRadiossStep01.php>
8. OpenRadioss cloud solver - Cloud HPC, <https://cloudhpc.cloud/openradioss-cloud-solver/>
9. Plans for topology optimization? · OpenRadioss · Discussion #616 - GitHub, <https://github.com/orgs/OpenRadioss/discussions/616>
10. pyMOTO: Modular framework for topology optimization with semi-automatic derivatives, https://www.researchgate.net/publication/369094972_pyMOTO_Modular_framework_for_topology_optimization_with_semi-automatic_derivatives
11. OpenRadioss/HOWTO.md at main - GitHub, <https://github.com/OpenRadioss/OpenRadioss/blob/main/HOWTO.md>
12. OpenRadioss on AWS®: HPC Workload Testing - Intel, <https://cdrdv2-public.intel.com/816822/Intel-AWS-instances-accelerate-OpenRadioss-HPC-simulations-methodology.pdf>
13. OpenRadioss/RELEASES.md at main - GitHub, <https://github.com/OpenRadioss/OpenRadioss/blob/main/RELEASES.md>
14. How to use OpenRadioss - Page 4 — Forum - Mecway, <https://mecway.com/forum/discussion/1531/how-to-use-openradioss/p4>
15. OpenRadioss Dataset Generation — NVIDIA PhysicsNeMo Framework, https://docs.nvidia.com/physicsemo/latest/physicsemo/examples/structural_mechanics/openradioss_dataset_gen/README.html
16. Checking interest for Python interface to starter/engine keywords · OpenRadioss · Discussion #678 - GitHub, <https://github.com/orgs/OpenRadioss/discussions/678>
17. Community Developed Tools - OpenRadioss, <https://openradioss.org/community-developed-tools/>
18. GitHub - Vortex-CAE/Vortex-Radioss: An open-source python library for processing Radioss results files, <https://github.com/Vortex-CAE/Vortex-Radioss>
19. Vortex-Radioss - D3plot convertor and open-source Python post-processing library for OpenRadioss - now released #2361 - GitHub, <https://github.com/orgs/OpenRadioss/discussions/2361>
20. Continuous Integration Testing - OpenMDAO, https://openmdao.org/newdocs/versions/latest/other_useful_docs/developer_docs/ci_testing.html
21. Calculation of Sensitivity Derivatives in an MDAO Framework - OpenMDAO, https://openmdao.org/pubs/Moore-2012-Calculation_of_Sensitivity_Derivatives_in_an_MDAO_Framework.pdf
22. Multidisciplinary design optimization framework with coupled derivative computation for hybrid aircraft - OpenMDAO, https://openmdao.org/pubs/joa_asquegli_2020.pdf
23. File Wrapping — OpenMDAO, https://openmdao.org/newdocs/versions/latest/other_useful_docs/file_wrap.html

24. ExternalCodeComp - OpenMDAO, https://openmdao.org/newdocs/versions/latest/features/building_blocks/components/external_code_comp.html
25. CarCrashNet: A Large-Scale Dataset and Hierarchical Neural Solver for Data-Driven Structural Crash Simulation - arXiv, <https://arxiv.org/html/2605.07098v2>
26. How to use an external code component within the optimization framework - Stack Overflow, <https://stackoverflow.com/questions/49070919/how-to-use-an-external-code-component-within-the-optimization-framework>
27. Model-Based Systems Analysis & Engineering (MBSA&E) for the Sustainable Flight National Partnership (SFNP) - OpenMDAO, https://openmdao.org/wp-content/uploads/2022/11/1045_MBSAE_OpenMDAO_Workshop_Final.pdf
28. Models - OpenRadioss, <https://openradioss.org/models/>
29. Compiling DAKOTA on Linux - foamBuilder, <https://www.foambuilder.com/pages/DAKOTASep02.php>
30. Coupling Dakota to a Simulation — dakota 6.19.0 documentation, https://snl-dakota.github.io/docs/6.19.0/users/usingdakota/introduction/couplingto_simulations.html
31. "Getting Started" Examples - Dakota Documentation, <https://snl-dakota.github.io/docs/6.18.0/users/usingdakota/examples/gettingstarted.html>
32. Dakota-SAS4A/SASSYS-1 Coupling for Uncertainty Quantification and Optimization Analysis, https://www.kns.org/files/int_paper/paper/MC2017_2017_5/P223S05-01ZhangG.pdf
33. Application of sensitivity analysis in DYMOND/Dakota to fuel cycle transition scenarios, https://www.epj-n.org/articles/epjn/full_html/2021/01/epjn210022/epjn210022.html
34. An airfoil with its main structural component, the wing box (marked in... - ResearchGate, https://www.researchgate.net/figure/An-airfoil-with-its-main-structural-component-the-wing-box-marked-in-bold-and-two_fig5_360419897
35. pyMOTO | Modular topology optimization in Python — pyMOTO 2.0.1 documentation, <https://pymoto.readthedocs.io/>
36. PyLCSS - KUTAY DEMIR, <https://kutaydemir.com.tr/pylcss/>
37. KUTAY DEMIR, <https://kutaydemir.com.tr/>
38. GitHub - kutaydemir07/PyLCSS: Python Low-Code System Solutions, <https://github.com/kutaydemir07/PyLCSS>
39. Cloud HPC CAE - CFD FEM FEA analyses for your simulations, <https://cloudhpc.cloud/>
40. Running Camry Impact Model Using Open Radioss and Apptainer - CIQ, <https://ciq.com/blog/running-camry-impact-model-in-ls-dyna-format-using-openradioss-and-apptainer>

41. PLAID: A Unified Data Model for Machine Learning on Heterogeneous Physics Simulations, <https://arxiv.org/html/2505.02974v3>
42. Advanced Mass Scaling (/AMS) - OpenRadioss - Confluence, <https://openradioss.atlassian.net/wiki/spaces/OPENRADIOSS/pages/9011303/Blow+Molding+with+AMS>